> **Note**
>
> - 设计并实现一个系统（可以是内核模块、用户态库或运行时库），能够监控LLM推理进程对KV缓存的访问模式（如访问频率、最近访问时间等），并以此定义每个KV缓存块的热度。
> - 运行时库：黑盒监控，你在底层拦截内存地址，猜测里面发生了什么
> - 用户态库：白盒监控，成为推理引擎的一部分

在当代大规模语言模型（LLM）的推理架构中，显存容量已取代计算峰值成为限制吞吐量和上下文长度的核心瓶颈 。KV缓存（Key-Value Cache）作为推理过程中保存历史注意力状态的动态结构，其规模随序列长度和批处理大小呈线性增长，往往占据了GPU显存的30%以上 。vLLM框架通过引入页式注意力（PagedAttention）机制，借鉴操作系统虚拟内存的设计思想，将KV缓存划分为固定大小的物理块，从而将显存利用率提升至96%以上 。然而，随着长文本需求及多轮对话业务的普及，简单的页式管理已不足以支撑复杂的显存调度需求。设计并实现一套能够深度监控KV缓存访问模式，并据此定义“块热度”的系统，对于实现高效的缓存替换（Eviction）、跨级卸载（Offloading）及前缀重用（Prefix Caching）具有至关重要的架构价值 。

Vllm现状

- **`vllm:kv_cache_usage_perc`**： KV 总体占用率
- **`vllm:prefix_cache_queries`**： 本地 prefix cache 查询 token 数
- **`vllm:prefix_cache_hits`**： 本地 prefix cache 命中 token 数
- **`vllm:external_prefix_cache_queries`**： 外部 prefix cache 查询 token 数
- **`vllm:external_prefix_cache_hits`**： 外部 prefix cache 命中 token 数
- **`vllm:prompt_tokens_cached`**： 缓存命中的 prompt token 总数
- **`vllm:prompt_tokens_recomputed`**： 缓存后又被重算的 token 数
- **`vllm:prompt_tokens_by_source{source}`**： prompt token 来源拆分
- **`vllm:request_prefill_kv_computed_tokens`**： 请求级 prefill 新算的 KV token 数
- **`vllm:kv_block_lifetime_seconds`**： KV block 生命周期
- **`vllm:kv_block_idle_before_evict_seconds`**： KV block 驱逐前 idle 时间
- **`vllm:kv_block_reuse_gap_seconds`**： KV block 复用间隔
- **`vllm:cache_config_info`**： cache 配置信息
- 能抓到：
  - `kv_cache_usage_perc`
  - `prefix_cache_queries`
  - `prefix_cache_hits`

    - `external_prefix_cache_queries`
    - `external_prefix_cache_hits`
    - `prompt_tokens_cached`
    - `prompt_tokens_recomputed`
    - `prompt_tokens_by_source`
    - `request_prefill_kv_computed_tokens`
    - `cache_config_info`
- 有条件能抓到：
    - `kv_block_lifetime_seconds`
    - `kv_block_idle_before_evict_seconds`
    - `kv_block_reuse_gap_seconds`
    - 前提是 开启 **`kv_cache_metrics`**

FlexKV现状

python侧监控的指标：

- `flexkv_py_cache_hit_blocks_total`： 按设备统计的缓存命中 block 总数
- `flexkv_py_cache_miss_blocks_total`： 缓存未命中的 block 总数（所有缓存层级都未命中）
- `flexkv_py_transfer_blocks_total`： 按传输类型和操作统计的传输 block 总数
- `flexkv_py_transfer_ops_total`： 按传输类型和操作统计的传输操作总数
- `flexkv_py_transfer_bytes_total`： 按传输类型和操作统计的传输字节总数
- `flexkv_py_mempool_total_blocks`： 按设备统计的内存池 block 总量
- `flexkv_py_mempool_free_blocks`： 按设备统计的内存池空闲 block 数量
- `flexkv_py_evicted_blocks_total`： 按设备统计的被淘汰 block 总数
- `flexkv_py_allocated_blocks_total`： 按设备统计的已分配 block 总数
- `flexkv_py_allocation_failures_total`： 按模式（全局 / 本地）统计的分配失败次数

C++侧监控的指标，主要是radixTree的：

- **`flexkv_cpp_cache_ops_total`**： 统计缓存操作次数
- **`flexkv_cpp_cache_blocks_total`**： 统计缓存操作涉及的block数量

开发方式对比

使用Monkey Patching方案（推荐）

KV 缓存的生命周期主要由 Block Manager（块管理器） 控制。在 vLLM 中，你需要监控的核心类通常是 `BlockManager` 或 `KVCacheManager`。

你需要监控的“埋点”主要有三个：

1. 分配（Allocate）：什么时候分配了新的 KV Block?
2. 读取/访问（Access）：Attention 计算时，读取了哪些 Block?
3. 释放（Free/Evict）：什么时候 Block 被释放了？

- **核心拦截点**:
  - `allocate_block()`：在新块分配时，初始化其热度计数器。
  - `read_block(block_id)`：在每次读取缓存块时，递增其访问计数，并更新最后访问时间戳。
  - `evict_block(block_id)`：在块被驱逐时，记录其最终的热度状态和驱逐原因。

创建一个标准的 Python 包结构：

```
kv_watcher/
├── __init__.py      # 入口文件，负责启动监控
├── monitor.py       # 核心监控逻辑
└── stats.py         # 存储热度数据的数据结构
```

实现监控逻辑(monitor.py)：

使用“装饰器模式”或“函数替换”来包裹原始函数。

```python
import time
import torch
from collections import defaultdict

class KVCacheMonitor:
    def __init__(self):
        # 记录每个 Block 的热度数据
        # 结构: {block_id: {"access_count": int, "last_access_time": float}}
        self.block_stats = defaultdict(lambda: {"access_count": 0, "last_access_time": 0})
        self.total_allocations = 0

    def on_block_allocated(self, block_id, gpu_ptr):
        """当分配 Block 时调用"""
        self.total_allocations += 1
        print(f"[Monitor] Block {block_id} allocated at {hex(gpu_ptr)}")

    def on_block_accessed(self, block_id):
        """当访问 Block 时调用（计算热度）"""
        self.block_stats[block_id]["access_count"] += 1
        self.block_stats[block_id]["last_access_time"] = time.time()

        # 这里可以添加实时告警逻辑
        # if self.block_stats[block_id]["access_count"] > 100: ...

    def on_block_freed(self, block_id):
        """当释放 Block 时调用"""
        # 注意：释放后不要立即删除记录，可能还需要统计历史热度
        print(f"[Monitor] Block {block_id} freed.")

# 全局单例
monitor = KVCacheMonitor()
```

注入代码(init .py)

```python
import importlib
from .monitor import monitor

def patch_vllm():
    try:
        # 1. 导入 vLLM 的核心模块
        # 注意：具体路径取决于 vLLM 版本，这里以常见路径为例
        block_manager_module = importlib.import_module("vllm.core.block_manager")

        # 2. 找到负责分配/访问的函数
        # 假设这是分配物理块的函数
        original_allocate = block_manager_module.BlockManager.allocate

        # 3. 定义包装函数
        def wrapped_allocate(self, num_blocks=1):
            # --- 监控逻辑开始 ---
            block_id = self._get_next_block_id() # 假设有个方法获取ID
            # 调用原始函数
            result = original_allocate(self, num_blocks)
            # 通知监控器
            monitor.on_block_allocated(block_id, result)
            # --- 监控逻辑结束 ---
            return result

        # 4. 替换 (Monkey Patch)
        block_manager_module.BlockManager.allocate = wrapped_allocate
        print("[KV Watcher] Successfully patched vLLM BlockManager!")

    except ImportError:
        print("[KV Watcher] vLLM not found, monitoring skipped.")

# 库加载时自动执行
patch_vllm()
```

上面的例子监控了分配。要监控访问频率（即哪些 Block 正在被计算），你需要深入到 `Attention` 算子中。

在 vLLM 中，Attention 计算通常由 `forward` 函数触发。你可以 Hook 这个函数：

```python
# 在 patch_vllm 中继续添加
model_runner_module = importlib.import_module("vllm.worker.model_runner")
original_forward = model_runner_module.ModelRunner.forward

def wrapped_forward(self, *args, **kwargs):
    # 获取当前的 block_table (vLLM 中记录了请求ID到物理BlockID的映射)
    # 这通常在 seq_group_metadata 或执行元数据中
    # 伪代码:
    # block_table = kwargs.get('block_table')
    # if block_table:
    #     for block_id in block_table:
    #         monitor.on_block_accessed(block_id)

    return original_forward(self, *args, **kwargs)

model_runner_module.ModelRunner.forward = wrapped_forward
```

库开发完了，如何使用这个库：

```bash
# 方式 A: 在 Python 脚本第一行导入
python -c "import kv_watcher; import run_vllm_server"

# 方式 B: 设置环境变量自动导入 (推荐)
export PYTHONPATH=/path/to/your/kv_watcher:$PYTHONPATH
python -c "import kv_watcher"
python run_vllm_server.py
```

基于 LD_PRELOAD 的动态链接库劫持（不推荐）

Linux 动态链接器允许你通过 `LD_PRELOAD` 环境变量，在程序启动时优先加载你的库。当程序调用标准函数（如 `malloc`）时，系统会先调用你库里的同名函数。

编写拦截代码

```c
// my_monitor.c
#define _GNU_SOURCE
#include <dlfcn.h>
#include <stdio.h>
#include <stdlib.h>

// 1. 定义原始 malloc 的函数指针
static void* (*real_malloc)(size_t) = NULL;

// 2. 构造函数：库加载时自动执行，获取原始函数地址
__attribute__((constructor))
void init_monitor() {
    real_malloc = dlsym(RTLD_NEXT, "malloc");
    fprintf(stderr, "[Monitor] KV Cache Runtime Loaded!\n");
}

// 3. 拦截 malloc
void* malloc(size_t size) {
    // 在这里插入你的逻辑：记录分配大小、调用栈、时间戳
    fprintf(stderr, "[Monitor] Allocating %zu bytes\n", size);

    // 调用真实的 malloc
    return real_malloc(size);
}
```

编译为动态库： `gcc -shared -fPIC -o my_monitor.so my_monitor.c -ldl`
运行为目标程序： `LD_PRELOAD=./my_monitor.so python run_llm.py`

这里需要拦截的是CUDA的API，例如cudaMalloc，cudaMemcpy。

这里的难点是从开发的视角只能看到内存地址和字节数，无法直接知道这一块内存是KVCache，更不知道它属于第几层，或者哪一个token。不推荐。

CUDA拦截和CUPTI（不推荐）

使用 NVIDIA 提供的 CUPTI (CUDA Profiling Tools Interface) 库。它是一个动态库，允许你注册回调函数，监听 CUDA 驱动 API 的活动。

缺点同上，不推荐。

内核模块方案（不考虑，太复杂）

可观测性建设（需要暴露出来哪些指标）

热度计算指标 (底层原子指标)

- 缓存块访问计数 (`kv_block_access_count_total`)
  - 类型: Counter
  - 定义: 每个KV缓存块被访问的总次数。这是衡量“访问频率”最直接的指标。
  - 关键标签:
    - `block_id`: 缓存块的唯一标识符。
    - `layer_id`: Transformer的层ID。不同层的注意力模式可能不同。
    - `seq_id`: 序列ID，用于区分不同请求。
    - `access_type`: `prefill` (预填充阶段) 或 `decode` (解码阶段)。解码阶段的访问模式更具时序性。
  - 价值: 这是计算任何复杂热度模型的基础。通过 `rate(kv_block_access_count_total[1m])` 可以计算出每个块的瞬时访问频率。
- 缓存块最后访问时间戳 (`kv_block_last_access_timestamp_seconds`)
  - 类型: Gauge
  - 定义: 记录每个KV缓存块最后一次被访问的Unix时间戳。这是衡量“最近访问时间 (Recency)”的关键。
  - 关键标签: 同上 (`block_id`, `layer_id`, `seq_id`)。
  - 价值: 结合当前时间，可以轻松计算出每个块的“空闲时间”。长时间未被访问的块是首要的“冷数据”候选者。
- 缓存块访问序列 (`kv_block_access_sequence`)
  - 类型: Histogram (用于统计) 或 通过日志记录
  - 定义: 记录在单个推理步骤（step）内，被访问的缓存块的序列模式。例如，是顺序访问还是跳跃访问。
  - 关键标签: `seq_len_bucket` (序列长度分段)。
  - 价值: 帮助理解访问的局部性。如果访问模式高度局部化，可以优化预取策略。

热度分布指标 (中层聚合指标)

- 热度等级分布 (`cache_hotness_tier_distribution`)
  - 类型: Gauge
  - 定义: 当前处于“热”、“温”、“冷”三个等级的缓存块数量或占比。这个分级是基于你自定义的热度算法（例如，结合访问频率和最近访问时间的加权分数）。
  - 关键标签: `hotness_tier` (hot/warm/cold), `storage_tier` (gpu_hbm/host_memory/swap)。

    - 价值: 直观展示你的热度定义是否合理，以及缓存数据在三级存储中的分布是否健康。例如，如果“热”数据过多导致GPU显存爆满，就需要调整热度阈值。
- 跨层热度关联 (`cross_layer_hotness_correlation`)
    - 类型: Gauge
    - 定义: 统计在同一个序列中，不同Transformer层的相同位置（block_id）的缓存块是否倾向于同时变热或变冷。
    - 关键标签: `layer_pair` (例如 `layer_0-1`, `layer_0-2`)。
    - 价值: 这是一个高级洞察。如果发现跨层热度高度相关，可以批量迁移整个序列在所有层的缓存块，减少管理开销。

热度效用指标 (顶层业务指标)

这些指标用于衡量你的“热度感知”系统对最终业务目标的影响。

- 分层缓存命中率 (`tiered_cache_hit_ratio`)
    - 类型: Counter
    - 定义: 分别计算从GPU（热）、内存（温）、外存（冷）命中的缓存访问次数。
    - 关键标签: `hit_tier` (hot/warm/cold)。
    - 价值: 这是验证热度定义有效性的黄金指标。一个成功的策略应保证 `hit_tier=hot` 的比率极高。
- 热度驱动的迁移效率 (`hotness_driven_migration_efficiency`)
    - 类型: Histogram
    - 定义: 记录每次因热度变化而触发的缓存迁移操作所花费的时间。
    - 关键标签: `migration_type` (hot_to_warm, warm_to_cold, cold_to_hot)。
    - 价值: 评估你的迁移策略本身带来的开销。特别是 `cold_to_hot` (预取) 的耗时，直接影响推理延迟。
- 将所有计算出的热度指标通过一个Prometheus Exporter暴露出来。
- 在Grafana中创建专门的“KV缓存热度仪表盘”，包含：
    - 实时热度分布图。
    - 单个序列的热度随时间变化曲线。
    - 热度驱动的迁移事件流。